SteadyThought: Mitigating LLM Under-Thinking via Thought-Level Preference Optimization

Zhenyue Peng[1,2] **Lin Yang**[3*] **Jie Wang**[4†] **Xiaoqi Ni**[2]
Hanzhu Chen[2] **Xize Liang**[2] **Yanjun Wu**[1,2]
Rujing Wang[1,2] **Kaiwen Zhou**[5] **Jianye Hao**[6]

[1]Hefei Institutes of Physical Science, Chinese Academy of Sciences, Hefei 230031, China
[2]University of Science and Technology of China, Hefei 230026, China
[3]Shanghai Artificial Intelligence Laboratory, Shanghai, China
[4]MoE Key Laboratory of Brain-inspired Intelligent Perception and Cognition,
University of Science and Technology of China
[5]Huawei Technologies Co., Ltd.
[6]College of Intelligence and Computing, Tianjin University
pzysa22168242@mail.ustc.edu.cn

Abstract

Flexible switching between reasoning trajectories (i.e., thought switching) has significantly enhanced the reasoning capabilities of Large Reasoning Models (LRMs). However, existing models often switch excessively yet fail to sustain promising reasoning thoughts——a phenomenon termed "under-thinking". While recent efforts suppress switching to mitigate this, such over-correction may discard valuable trajectories. To address this challenge, we propose **Steady Thought** (**ST**), a novel thought-level preference optimization framework. ST formalizes under-thinking as a preference issue at switching points, forcing single-path continuations from identified points to yield high-quality reasoning. Then, ST performs thought-level preference optimization by treating the newly generated response as preferred and the original one as dis-preferred. Experiments across multiple models and datasets show that ST effectively reduces token consumption, achieving reductions of up to 63.8%, while maintaining strong reasoning accuracy. Further analysis suggests that ST helps models acquire a generalizable ability to reduce redundant switching across domains and languages.

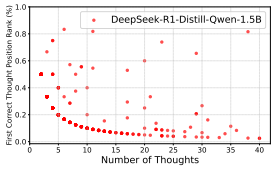
1 Introduction

Recent advancements in Large Reasoning Models (LRMs), such as DeepSeek-R1 [DeepSeek-AI et al., 2025] and GPT-o1 [OpenAI, 2024], have demonstrated remarkable capabilities in solving complex tasks. Their success is largely attributed to the emulation of human-like "slow thinking" processes, specifically the ability to flexibly switch between reasoning strategies—a mechanism termed *thought switching* [Zeng et al., 2025, Muennighoff et al., 2025, Yang et al., 2025, Chen et al., 2025b]. This adaptability allows models to explore diverse reasoning paths, theoretically leading to more accurate and robust inferences.

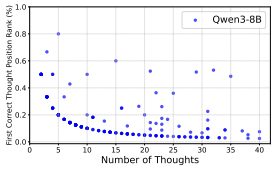
However, recent studies have shown that LRMs tend to switch thoughts too frequently [Wang et al., 2025a, Ding et al., 2025, Chen et al., 2025a] and often fail to follow the promising thoughts—a phenomenon termed "under-thinking" [Wang et al., 2025c]. Specifically, Figures 1a and 1b

*Project lead.
†Corresponding author: jiewangx@ustc.edu.cn.

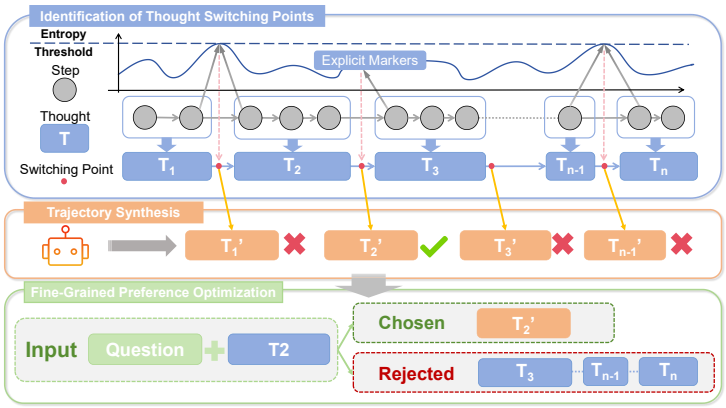
40th Conference on Neural Information Processing Systems (NeurIPS 2026).



(a) DeepSeek-R1-Distill-Qwen-1.5B on MATH-500



(b) Qwen3-8B on AIME2024



(c) Overview of Steady Thought framework

Figure 1: (a) Rank of the first correct thought position for DeepSeek-R1-Distill-Qwen-1.5B on MATH-500. (b) Rank of the first correct thought position for Qwen3-8B on AIME2024. (c) Overview of the ST framework, which identifies thought switching points using two complementary signals: explicit linguistic markers and concurrent entropy spikes around step boundaries. Starting from these identified junctions, the model is guided via logit control to generate persistent reasoning continuations, effectively verifying the potential of the current thought. These synthesized paths are then used to construct fine-grained preference pairs for the STPO objective, which reinforces trajectory commitment at the exact point of divergence to effectively mitigate under-thinking and improve reasoning efficiency.

illustrate the initial emergence of correct thoughts during the models' reasoning processes. They reveal that the models often find a reliable thought early (which can lead to the correct answer), but still proceed with numerous additional thought switches. This issue stems from the model's inability to recognize and commit to high-potential thoughts, effectively hindering the depth and coherence of reasoning. Furthermore, such erratic switching results in substantial inefficiency, wasting computational resources on unproductive generation.

To mitigate the shallow reasoning resulting from frequent thought switching, existing approaches primarily focus on indiscriminately suppressing switching behaviors. For example, some methods [Wang et al., 2025c,a, Ding et al., 2025] assume that the thought switching often starts with certain special tokens (e.g., "alternatively" or "wait") and discourage the model from switching by lowering the probabilities of these tokens during decoding or reducing their rewards in training. On the other hand, Chen et al. [2025a] take a different approach by operating in the representation space: they steer the model's hidden states away from a learned "switching" vector and toward a "reasoning" vector, thereby suppressing switching behavior in a more structured manner. While these methods effectively reduce the frequency of switches, they apply suppression indiscriminately, thereby restricting the model's flexibility to explore alternative paths when the current trajectory is indeed incorrect. These limitations highlight the need for a more selective mechanism—one that preserves the ability to explore new reasoning thoughts when the current trajectory is unpromising, while encouraging deeper commitment to a thought when it shows promise.

To this end, we propose **Steady Thought (ST)**, a novel thought-level preference optimization framework for mitigating under-thinking by selectively discouraging thought switching, thereby improving token efficiency without sacrificing reasoning performance. We first formalize the under-thinking problem as a preference alignment issue at critical decision points. These points are effectively identified through a complementary approach that combines marker-based detection with the monitoring of entropy spikes at step boundaries. Subsequently, starting from the identified thought switching point, the model is guided via logit control to follow its preceding thought in a single direction without further switching until the final answer is reached. By evaluating these completions against correct answers, we obtain high-quality reasoning continuations that exhibit stable late-stage persistence, which serve as the preferred targets for optimization, and the original response—characterized by unnecessary switching—is treated as dis-preferred. Finally, we apply a thought-level preference optimization method named **Steady Thought Preference Optimization**

(STPO) to encourage the model to persist in promising reasoning directions while discouraging unnecessary switching at thought switching points.

Our main contributions are threefold:

- We analyze the specific manifestations of the under-thinking phenomenon and formalize it as a preference optimization problem.
- We propose **Steady Thought** (ST), a novel thought-level preference optimization framework that encourages models to develop and stick to high-potential reasoning thoughts without compromising their flexibility to explore alternative reasoning trajectories.
- We validate the effectiveness of ST through extensive experiments across multiple models and reasoning benchmarks, demonstrating substantial token reductions of up to 63.8% while maintaining strong reasoning accuracy.

2 Background

2.1 Problem Formulation

We formalize the reasoning process of a Large Reasoning Model (LRM) as a trajectory punctuated by multiple thought switching points, which naturally partitions the response y to a question x into a sequence of distinct thoughts:

$$y = (T_1, T_2, \ldots, T_n), \tag{1}$$

where each thought T_i represents a coherent segment of reasoning. Following a sequence of reasoning explorations to reach the i-th thought, denoted by the prefix $\mathbf{P}_i = (\mathbf{x}, T_1, \ldots, T_i)$, the policy π_θ defines a conditional probability distribution over all possible subsequent trajectories.

The problem of **under-thinking** arises at a critical decision point. After the model generates a **promising thought** T_i (one that *can* lead to a correct answer), it faces a choice. Within the set of possible next trajectories, we are interested in two specific types:

- The **Commit Trajectory** ($\tau_\mathbf{c}$): The ideal trajectory where the model commits to and correctly completes the promising thought T_i. We denote this completed sequence as T_i'.
- The **Switch Trajectory** ($\tau_\mathbf{s}$): The suboptimal trajectory observed in the data, where the model abandons T_i and switches to a new, often wasteful line of reasoning $(T_{i+1}, \ldots, T_n)$.

Our objective is to align the model's policy π_θ with the preference for the commit trajectory over the switch trajectory, i.e., $\tau_\mathbf{c} \succ \tau_\mathbf{s}$. To formalize this, we define a latent **Steadiness Score** $S_\pi(\tau|\mathbf{P}_i)$, which quantifies the policy's inclination towards a specific trajectory τ given the prefix $\mathbf{P}_i$. The preference $\tau_\mathbf{c} \succ \tau_\mathbf{s}$ implies that the score of the commit trajectory should be higher.

Following the Bradley-Terry model, the probability of this preference can be expressed through the difference in scores:

$$P(\tau_\mathbf{c} \succ \tau_\mathbf{s}|\mathbf{P}_i) = \sigma(S_\pi(\tau_\mathbf{c}|\mathbf{P}_i) - S_\pi(\tau_\mathbf{s}|\mathbf{P}_i)), \tag{2}$$

where σ is the sigmoid function. This abstract scoring function provides a powerful new lens through which to view the problem. Crucially, it can be directly instantiated by the LRM's own log-probabilities. Preference optimization methods like Direct Preference Optimization (DPO) [Rafailov et al., 2024] and Simple Preference Optimization (SimPO) [Meng et al., 2024] allow us to directly optimize the policy π_θ such that its implicit scoring function, where $S_\pi(\tau|\mathbf{P}_i) := \log \pi_\theta(\tau|\mathbf{P}_i)$, satisfies the desired preference relationship. This provides a principled way to train the model to be a more steadfast and effective reasoner.

2.2 Preference Optimization Methods

DPO significantly advanced the alignment of language models with human preferences by leveraging the Bradley-Terry preference model to bypass the complex reward modeling and policy optimization of traditional reinforcement learning. However, DPO is sensitive to response-length differences because its sequence-level log-probability ratios accumulate token-level contributions, making the preference signal dependent on response length. SimPO addresses this limitation by introducing a length-normalized reward with a target margin, effectively mitigating length bias without requiring a

reference model. Given the preference data $\mathcal{D} = \{(\mathbf{x}, \mathbf{y}_w, \mathbf{y}_l)\}$, where $\mathbf{x}$ is the input prompt, $\mathbf{y}_w$ is the preferred response, and $\mathbf{y}_l$ is the dispreferred response. SimPO optimizes the following objective:

$$\mathcal{L}_{\text{SimPO}}(\pi_\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\frac{\beta}{|y_w|}\log\pi_\theta(y_w \mid x) - \frac{\beta}{|y_l|}\log\pi_\theta(y_l \mid x) - \gamma\right)\right]. \quad (3)$$

where σ is the sigmoid function, with β as a temperature parameter controlling sensitivity to preference differences and γ as the target reward margin for the model to achieve.

3 Method

We propose Steady Thought (ST), a preference optimization framework designed to encourage reasoning persistence in promising reasoning thoughts. As illustrated in Figure 1c, ST first identifies thought switching points and then synthesizes persistent trajectories from these points. Finally, it applies fine-grained preference optimization to reinforce commitment to promising thoughts and reduce unnecessary switching.

3.1 Identification of Thought Switching Points

To identify points at which a reasoning trajectory changes direction, we propose a complementary dual-channel detection framework. We first partition each model response into a sequence of candidate steps using the structural delimiter "`\n\n`". Each step boundary is then examined using two complementary signals:

- **Explicit Linguistic Markers:** The subsequent step begins with a predefined phrase that explicitly signals a change in reasoning direction. Representative markers include "alternatively" and "another way," with the full list provided in Appendix B.2.
- **Boundary Entropy Spikes:** High entropy values (exceeding a threshold τ) are observed concurrently at the terminal tokens of the preceding step and the initial tokens of the next step. At each token position, the entropy of the model's predictive distribution is computed as

$$H(P) = -\sum_{v\in\mathcal{V}} P(v)\log P(v), \quad (4)$$

  where $\mathcal{V}$ denotes the vocabulary and $P(v)$ is the predicted probability of token v at that position.

To operationalize entropy-based switching detection, we calibrate an entropy threshold τ using responses sampled from the training pool. For each model, we select the lowest candidate threshold at which at least 80% of the detected boundaries correspond to genuine switches in reasoning direction, thereby balancing detection reliability and boundary coverage. The calibrated threshold is then fixed and applied to the full training set for thought switching point detection. Given this threshold, a candidate step boundary satisfies the entropy criterion when predictive entropy simultaneously exceeds τ on both sides of the boundary, at the end of the preceding step and the beginning of the subsequent step. Further details of the calibration procedure and threshold sensitivity analysis are provided in Appendix F

A step boundary is identified as a **thought switching point** if either the linguistic-marker condition or the entropy criterion is satisfied. Each switching point marks the beginning of a new thought, and subsequent steps are grouped into the same thought until the next switching point is encountered. This process partitions the original response $\mathbf{y}$ into a structured sequence of thoughts:

$$\mathbf{y} = (T_1, T_2, \ldots, T_n). \quad (5)$$

The rationale for this dual-channel approach lies in the complementary nature of linguistic and stochastic signals. Explicit keywords are highly effective at capturing universal and structured switching points across different models. In contrast, some studies have indicated that high entropy of tokens serves as a signal for latent uncertainty in the model's predictive distribution, which usually occurs when the model is re-planning or exploring a new reasoning trajectory [Wang et al., 2025b,

Cui et al., 2025]. This allows for the detection of model-specific or problem-specific implicit switches that lack formal transition words. For example, when solving a mathematical problem concerning properties of regular n-gons under permutations, the model reaches an impasse after an extended but inconclusive qualitative analysis of inscribed angles and arcs. It then pivots to a coordinate-based analytical approach, stating, "Let me think about the coordinates...". Although this transition does not contain any of our predefined linguistic markers, it is accompanied by a pronounced entropy spike around the step boundary, reflecting the shift from qualitative geometric reasoning to a different analytical strategy. By combining the two channels, our method captures both explicit linguistic shifts and implicit uncertainty-associated transitions. We provide an ablation study of the two detection channels in Appendix E.

3.2 Trajectory Synthesis at Switching Points

Once a thought switching point is identified, we must determine whether the model should have persisted with the current reasoning thought or transitioned to an alternative one, as exhibited in the original response. We evaluate the potential of each segmented thought T_i by generating a continuation from the question Q and the current thought itself. Since switching points are used to partition the trajectory into localized reasoning units, this setup allows us to directly assess whether T_i can support a correct continuation. It also reduces the context length during trajectory synthesis, allowing more thought-level instances to be retained under limited memory and context budgets.

To ensure the model remains committed to the current trajectory, we apply logit control during the decoding phase. Specifically, we sharply decrease the logit values of transition tokens that typically trigger a switch, such as "wait", "but" or "however". By driving the selection probability of these tokens toward zero, we force the model to pursue the current line of reasoning without further deviation until a final answer is generated. This synthesized completion is denoted as:

$$T_i' \sim \pi_\theta(\cdot \mid Q, T_i), \tag{6}$$

where T_i' represents the model's self-generated reasoning path under the constraint of reasoning persistence. We retain T_i as a promising thought only if the corresponding T_i' reaches the correct final answer; otherwise, it is discarded. Further analysis of the synthesis cost and data utilization is provided in Appendix G.

3.3 Fine-Grained Preference Optimization

A significant limitation of holistic preference optimization is that it treats entire reasoning chains as monolithic blocks. For complex multi-step problems, an incorrect response often contains a substantial sequence of valid initial reasoning. Rejecting such a response in its entirety discards these valuable correct segments and provides a noisy, unfocused learning signal.

To address this, our approach provides more granular supervision by focusing on the critical switching points where the reasoning diverges. We aim to teach the model to commit to a promising thought once it has been identified, rather than abandoning it. Specifically, an answer $\mathbf{y}$ can be decomposed into a sequence of thoughts $\mathbf{y} = (T_1, \ldots, T_n)$. When we identify a promising thought T_i, the optimization then hinges on the trajectory the model takes from this point forward, conditioned on both the original question Q and the thought T_i itself.

The preference pair is constructed based on the continuations from this shared context:

$$\begin{aligned} \text{Chosen:} \quad & \mathbf{y}_w = T_i' && \text{(The full, correct completion)} \\ \text{Rejected:} \quad & \mathbf{y}_l = (T_{i+1}, T_{i+2}, \ldots, T_n) && \text{(The subsequent wasteful thoughts)} \end{aligned}$$

Inspired by the reference-free and length-normalized objective of SimPO , the STPO loss is formulated as:

$$\mathcal{L}_{\text{STPO}}(\pi_\theta) = -\mathbb{E}_{(Q,T_i,\mathbf{y}_w,\mathbf{y}_l)\sim\mathcal{D}}\Big[\log\sigma\Big(\frac{\beta}{|\mathbf{y}_w|}\log\pi_\theta(\mathbf{y}_w|Q,T_i) - \frac{\beta}{|\mathbf{y}_l|}\log\pi_\theta(\mathbf{y}_l|Q,T_i) - \gamma\Big)\Big]. \tag{7}$$

This formulation provides a crucial distinction from conventional preference tuning. The learning signal, embodied by the conditional log-probabilities $\log\pi_\theta(\cdot|Q,T_i)$, is applied directly at the point of divergence. It forces the model to learn not just what a good final answer looks like, but to recognize and commit to a promising intermediate thought. This targeted, thought-level supervision is key to effectively mitigating the model's tendency to "under-think" and abandon viable reasoning paths.

Table 1: Experimental results on four large reasoning models.

Method	MATH-500		AIME 2024		GSM8K		LiveCode		Overall	
	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓
DeepSeek-R1-Distill-Qwen-1.5B										
Vanilla	82.0	4385	27.5	11273	81.9	1448	30.3	9623	55.43	6682
NoThink	65.8	749	8.7	3185	53.6	263	20.7	813	37.20 (-18.23)	1252 (-81.3%)
NOWAIT	81.0	2590	22.1	6941	79.9	727	26.8	5122	52.45 (-2.98)	3845 (-42.4%)
SEAL	82.6	3252	25.4	9120	79.7	860	29.5	7948	54.30 (-1.13)	5295 (-20.8%)
TIP	82.2	4462	30.4	11037	81.6	1292	29.5	9574	55.93 (+0.50)	6591 (-1.3%)
Steady Thought	83.6	2691	30.8	7577	81.5	1118	33.2	7335	57.28 (+1.85)	4680 (-30.0 %)
Qwen3-8B										
Vanilla	91.4	4724	62.1	10895	95.6	1759	71.8	7112	80.23	6122
NoThink	85.2	933	25.8	3504	93.6	289	45.6	584	62.55 (-17.68)	1327 (-78.3%)
NOWAIT	92.8	3327	55.8	9371	95.5	1067	80.3	5285	81.10 (+0.87)	4762 (-22.2%)
SEAL	92.2	4034	58.8	10372	95.9	1421	83.4	6414	82.58 (+2.35)	5560 (-9.2%)
TIP	92.6	4516	58.3	10892	95.9	1705	81.1	7072	81.98 (+1.75)	6046 (-1.2%)
Steady Thought	93.4	2139	64.6	7640	95.7	637	83.2	5631	84.23 (+4.00)	4011 (-34.5 %)
DeepSeek-R1-Distill-Qwen-14B										
Vanilla	93.6	3349	60.4	8974	94.8	894	70.1	6789	79.73	5001
NoThink	41.7	824	27.1	3279	90.1	256	44.0	708	50.73 (-29.00)	1266 (-74.7%)
NOWAIT	89.8	2255	49.2	7030	93.7	692	70.8	4921	75.88 (-3.85)	3724 (-25.5%)
SEAL	92.6	3253	60.8	8831	94.7	880	75.1	6706	80.80 (+1.07)	4917 (-1.7%)
TIP	93.0	3098	62.5	9060	94.7	875	73.5	6958	80.93 (+1.20)	4998 (-0.1%)
Steady Thought	93.4	2457	64.6	7535	95.3	791	74.8	5361	82.03 (+2.30)	4036 (-19.3%)
Qwen3-32B										
Vanilla	94.0	4000	67.1	10087	95.7	1276	82.6	7361	84.85	5681
NoThink	93.2	2913	64.6	9772	93.6	479	58.4	272	77.45 (-7.40)	3359 (-40.9%)
NOWAIT	93.8	2982	67.5	8772	95.9	934	85.4	5990	85.65 (+0.80)	4670 (-17.7%)
SEAL	94.8	3470	73.3	9461	96.4	1078	85.9	6442	87.60 (+2.75)	5112 (-10.0%)
TIP	94.2	3951	67.5	10195	95.9	1271	82.0	7478	84.90 (+0.05)	5723 (+0.8%)
Steady Thought	94.8	2819	73.8	8225	96.6	844	85.9	6193	87.78 (+2.93)	4520 (-20.4%)

4 Experiments

4.1 Datasets

We selected the omni-math [Gao et al., 2024] dataset as the source of our training data. This dataset contains thousands of problems at the level of the International Mathematical Olympiad, which are categorized by difficulty. We sampled problems from various difficulty levels and used a target model (e.g., Qwen3-8B) to perform inference and generate responses.

We evaluate our method on four datasets: (1) MATH-500 [Hendrycks et al., 2021], which contains 500 math problems across diverse domains such as algebra, geometry, and number theory; (2) AIME 2024 [MAA Committees, 2024], a challenging math competition benchmark consisting of 30 problems from the American Invitational Mathematics Examination; (3) GSM8K [Cobbe et al., 2021], which contains 1,319 grade-school math word problems requiring multi-step reasoning; and (4) LiveCode [Jain et al., 2024], which evaluates coding ability with 400 programming problems collected from competitive programming platforms. Since our training data is drawn only from mathematical datasets, we use LiveCode as an out-of-distribution benchmark to evaluate generalization beyond math.

4.2 Baseline and Metrics

We compare Steady Thought against four base models (DeepSeek-R1-Distill-Qwen-1.5B [DeepSeek-AI et al., 2025], Qwen3-8B [Qwen Team, 2024], DeepSeek-R1-Distill-Qwen-14B [DeepSeek-AI et al., 2025] and Qwen3-32B [Qwen Team, 2024]), as well as four test-time efficiency methods, namely NoThink, NOWAIT, SEAL and TIP.

- NoThink: Ma et al. [2025] make the model skip the thinking process and output the response directly by adding the `<think>` tag after the `</think>` tag in the prompt.
- NOWAIT: Wang et al. [2025a] reduces the logits values of certain keywords that represent reflection (e.g., "wait") during decoding, making them almost impossible to output.
- SEAL: Chen et al. [2025a] first collects multiple responses and categorizes them into three types of thoughts: execution, reflection, and transition. A differential vector S is derived in the latent space between execution vectors and the combination of reflection/transition vectors. During decoding, the hidden states are modified as $\tilde{H} = H + \alpha \cdot S$, biasing the hidden layer toward execution-oriented thoughts.

Table 2: Experimental results on diverse OOD domains

Method	MuSR		StrategyQA		MedQA-USMLE		Overall	
	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓
DeepSeek-R1-Distill-Qwen-1.5B								
Vanilla	40.7	2590	58.7	538	29.3	2077	42.90	1735
NoThink	31.2	248	56.0	83	27.2	290	38.13 (-4.77)	207 (-88.1%)
NOWAIT	41.5	578	59.3	433	28.6	955	43.13 (+0.23)	655 (-62.2%)
SEAL	21.8	1424	57.7	469	24.4	1302	34.63 (-8.27)	1065 (-38.6%)
TIP	39.8	2518	58.4	526	28.8	1901	42.33 (-0.57)	1648 (-5.0%)
Steady Thought	46.3	592	59.7	414	31.9	1068	45.97 (+3.07)	691 (-60.2%)
Qwen3-8B								
Vanilla	58.9	2176	79.7	622	79.9	1936	72.83	1578
NoThink	55.7	818	78.1	313	61.4	741	65.07 (-7.76)	624 (-60.5%)
NOWAIT	60.7	1243	82.6	438	78.0	1341	73.77 (+0.94)	1007 (-36.2%)
SEAL	24.3	1906	57.4	545	24.8	1761	35.50 (-37.33)	1404 (-11.0%)
TIP	59.9	2147	79.8	612	80.2	1953	73.30 (+0.47)	1570 (-0.5%)
Steady Thought	59.3	1433	83.3	382	79.9	1295	74.17 (+1.34)	1036 (-34.3%)

- TIP: Wang et al. [2025c] reduces the logits of switching tokens within a local decoding window and restores their original probabilities outside the window until the next thought is reached, which encourages longer intervals between switching tokens.

We report the accuracy and average token count for each task. We average over eight test runs for AIME 2024 and two runs for LiveCode. Since MATH-500 and GSM8K contain larger test sets and exhibit relatively stable model outputs, we use a single test run for these two datasets.

4.3 Main Results

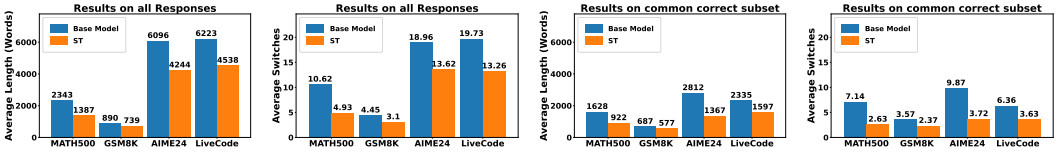
As shown in Table 1, Steady Thought consistently reduces reasoning token consumption across four models with different scales and architectures. Compared with the vanilla models, ST reduces the average token consumption by 30.0%, 34.5%, 19.3%, and 20.4% on the 1.5B, 8B, 14B, and 32B models, respectively. Importantly, these efficiency gains do not come at the expense of reasoning performance: ST achieves average accuracy improvements of 1.85, 4.00, 2.30, and 2.93 percentage points across the four models. Unlike methods that indiscriminately suppress thinking or switching behaviors, ST selectively discourages unnecessary switching, leading to more efficient reasoning without sacrificing overall performance.

Notably, LiveCode serves as an OOD benchmark because all models are trained only on mathematical data. On this coding benchmark, ST reduces token usage by 23.8%, 20.8%, 21.0%, and 15.9% across the four models, respectively. These efficiency gains are accompanied by accuracy improvements of 2.9, 11.4, 4.7, and 3.3 percentage points. The consistent improvements on an unseen domain indicate that the benefits of ST extend beyond the mathematical training distribution, suggesting that the learned behavior can generalize across reasoning domains. A trajectory comparison for the same task before and after ST training is provided in Appendix H.

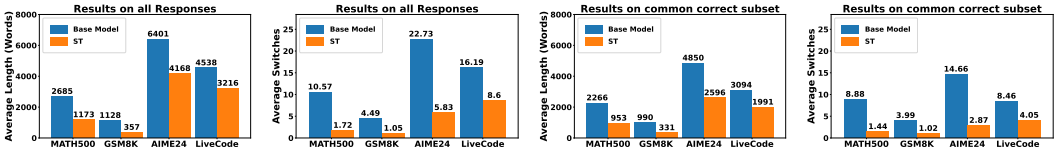
4.4 Broader Generality Across Diverse Reasoning Domains

To further evaluate the generalization ability of ST beyond mathematical and coding domains, we still use only mathematical data for training and introduce three out-of-distribution datasets from broader, less logic-intensive domains for evaluation: MuSR [Sprague et al., 2023] for multi-step soft reasoning, StrategyQA [Geva et al., 2021] for commonsense reasoning, and MedQA-USMLE [Jin et al., 2021] for medical question answering. We report the average over four evaluation runs for each dataset.

As shown in Table 2, ST substantially reduces token consumption across the three OOD datasets. For DeepSeek-R1-Distill-Qwen-1.5B, the average token consumption decreases by 60.2%, accompanied by an average accuracy improvement of 3.07 percentage points. For Qwen3-8B, ST reduces the average output length by 34.3% while improving the average accuracy by 1.34 percentage points. These results indicate that ST can still transfer to broader reasoning scenarios such as commonsense understanding, multi-hop reasoning, and domain-specific question answering. This further suggests that ST enables the model to acquire a transferable capability, rather than merely memorizing superficial patterns from the mathematical training data.



(a) DeepSeek-R1-Distill-Qwen-1.5B



(b) Qwen3-8B

Figure 2: Comparison of switching behavior across four metrics: average response length and number of switches on all responses (first two columns) and on the common correct subset (last two columns). Response length is measured in words. The first row shows DeepSeek-R1-Distill-Qwen-1.5B, and the second row shows Qwen3-8B. Blue bars represent the base model, and orange bars represent ST.

Table 3: Results on the controlled evaluation set.

Method	**Acc(%)↑**	**Token↓**
DeepSeek-R1-Distill-Qwen-1.5B		
Vanilla	13.2	7838
Steady Thought	20.3	5468
Qwen3-8B		
Vanilla	23.5	9830
Steady Thought	26.6	6783

Table 4: Results on GAOKAO-bench.

Method	**Acc(%)↑**	**Token↓**
DeepSeek-R1-Distill-Qwen-1.5B		
Vanilla	80.2	2829
Steady Thought	83.7	1746
Qwen3-8B		
Vanilla	86.0	4420
Steady Thought	88.4	1872

4.5 In-depth Analysis and Ablation Studies

4.5.1 Analysis of Thought Switching

ST encourages the model to persist when it reaches a promising thought, thereby reducing unnecessary switches and the resulting redundant generation. This effect is primarily evidenced by the model's ability to reach correct conclusions with significantly higher efficiency. To evaluate this structural optimization, we conducted a comparative analysis of the model's behavior before and after ST training across four dimensions: (1) average total output length on all responses, (2) the average number of thought switches on all responses, (3) output length within correct reasoning trajectories, and (4) the average number of switches within those correct trajectories. We measure response length by the number of words.

As illustrated in Figure 2, models after ST consistently exhibit a reduction in both total output length and the number of thought switches across all datasets. More importantly, this trend persists even for the same set of problems correctly solved both before and after training. By reaching the same correct conclusions with fewer thought-level transitions, the model demonstrates that it has learned to prune redundant reasoning branches and avoid unnecessary switching in its thought process. Consequently, these results provide strong evidence that the observed efficiency gains are driven by more direct reasoning trajectories, in which redundant thought switching is substantially reduced while correct solutions are preserved. We provide further analysis on more models in Appendix D.

4.5.2 Analysis of Recovery Capability

We further verify that ST does not compromise the model's existing ability to abandon an incorrect reasoning direction when a switch is needed. To this end, we construct a controlled evaluation set from several datasets such as MATH-500 and AIME 2024. We generate responses with the base model and use DeepSeek-V4-Pro to judge the correctness of the first thought, retaining only those with an incorrect initial reasoning direction. The resulting incorrect thoughts are used as fixed prefixes, from which the model continues generation. Token counts are computed only over the generated continuations.

As shown in Table 3, this capability is not impaired by ST. For DeepSeek-R1-Distill-Qwen-1.5B, accuracy increases from 13.2% to 20.3%, while the average continuation length decreases from 7,838 to 5,468 tokens. Similarly, for Qwen3-8B, accuracy increases from 23.5% to 26.6%, with continuation length decreasing from 9,830 to 6,783 tokens. These results indicate that ST, although reducing thought switching, does not simply enforce persistence in the current reasoning direction when that direction is incorrect. Instead, it preserves the models' ability to recover from an incorrect early thought while generating substantially shorter continuations.

4.5.3 Analysis of Cross-Lingual Generalization

To verify that ST does not merely teach the model to reduce the generation of the manually specified switching tokens used during the Trajectory Synthesis stage, we further evaluate its cross-lingual generalization ability. Specifically, we still construct preference data and train the model only on English mathematical datasets, but test it on the Chinese mathematical benchmark GAOKAO-bench Zhang et al. [2023].
The results are shown in Table 4. For DeepSeek-R1-Distill-Qwen-1.5B, ST reduces the average token count by 38.3% while improving accuracy from 80.2% to 83.7%. For Qwen3-8B, ST decreases the average token count by 57.6% while increasing accuracy from 86.0% to 88.4%. These results show that, even when the training and test data differ substantially in language form, ST can still improve accuracy while significantly reducing output length. This suggests that what the model learns through ST is not simply the suppression of certain English switching tokens, but a cross-lingually transferable reasoning behavior pattern.

4.5.4 Analysis of Different Training Methods

We compare different training strategies in the final stage of ST, as shown in Table 5. We consider two supervision granularities. In the standard setting, the input is Q, with $(T_1, \ldots, T_i, T_i')$ as the chosen target; preference-based methods additionally use the original trajectory $(T_1, \ldots, T_n)$ as the rejected response. In the thought-level setting, the input is (Q, T_i) and the chosen continuation is T_i'; DPO-thought and STPO additionally use $(T_{i+1}, \ldots, T_n)$ as the rejected continuation. Accordingly, we compare SFT-standard, SFT-thought, DPO-standard, SimPO-standard, DPO-thought, and STPO.

Table 5: Comparison of different training methods.

Training Method	MATH-500		AIME 2024	
	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓
DeepSeek-R1-Distill-Qwen-1.5B				
Vanilla	82.0	4385	27.5	11273
+ SFT-standard	80.4	2650	22.9	7169
+ SFT-thought	78.0	2661	20.8	6650
+ DPO-standard	82.6	4273	30.8	10701
+ DPO-thought	82.8	4428	26.7	11364
+ SimPO-standard	81.6	4049	29.2	10592
+ STPO	83.6	2691	30.8	7577
Qwen3-8B				
Vanilla	91.4	4724	62.1	10895
+ SFT-standard	91.8	3181	61.7	9381
+ SFT-thought	91.4	3020	60.8	8663
+ DPO-standard	91.8	4495	60.8	10812
+ DPO-thought	92.8	4594	60.0	10628
+ SimPO-standard	93.6	2908	60.8	9151
+ STPO	93.4	2139	64.6	7640

Since SFT learns only from chosen continuations, it mainly encourages imitation of target completions without explicitly distinguishing persistence from unnecessary switching. In contrast, preference optimization directly contrasts these behaviors at the thought-switching point, providing a more targeted signal for sustaining promising thoughts while discouraging unnecessary switches. However, due to our data construction, the chosen continuation is usually much shorter than the rejected one. Since DPO is sensitive to response-length differences, such differences can interfere with preference learning, limiting its effectiveness in reducing unnecessary switching. SimPO alleviates this issue through length-normalized rewards, but SimPO-standard still operates on the entire trajectory and thus provides only coarse-grained supervision. In contrast, STPO combines length normalization with thought-level optimization, reducing the influence of sequence-length signals and focusing the learning objective on the exact divergence point between persistence and unnecessary switching. As a result, STPO reduces token consumption by discouraging unnecessary switching behaviors while maintaining strong reasoning accuracy.

5 Conclusion

In this paper, we present Steady Thought (ST), a thought-level preference optimization framework for mitigating under-thinking in Large Reasoning Models. ST identifies thought switching points using complementary linguistic and entropy-based signals, synthesizes persistent continuations, and applies fine-grained preference optimization to discourage unnecessary switching while preserving useful reasoning flexibility. Experiments across multiple models and benchmarks show that ST substantially reduces token consumption while maintaining reasoning accuracy, with token reductions of up to 63.8%. Its consistent performance on out-of-distribution tasks further suggests that the learned behavior generalizes beyond the training domain. Overall, ST provides an effective approach to improving reasoning efficiency by reducing redundant thought switching.

References

Pranjal Aggarwal and Sean Welleck. L1: Controlling how long a reasoning model thinks with reinforcement learning, 2025. URL https://arxiv.org/abs/2503.04697.

Runjin Chen, Zhenyu Zhang, Junyuan Hong, Souvik Kundu, and Zhangyang Wang. Seal: Steerable reasoning calibration of large language models for free, 2025a. URL https://arxiv.org/abs/2504.07986.

Zhipeng Chen, Yingqian Min, Beichen Zhang, Jie Chen, Jinhao Jiang, Daixuan Cheng, Wayne Xin Zhao, Zheng Liu, Xu Miao, Yang Lu, Lei Fang, Zhongyuan Wang, and Ji-Rong Wen. An empirical study on eliciting and improving r1-like reasoning models, 2025b. URL https://arxiv.org/abs/2503.04548.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL https://arxiv.org/abs/2110.14168.

Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models, 2025. URL https://arxiv.org/abs/2505.22617.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha,

Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948.

Bowen Ding, Yuhan Chen, Futing Wang, Lingfeng Ming, and Tao Lin. Do thinking tokens help or trap? towards more efficient large reasoning model, 2025. URL https://arxiv.org/abs/2506.23840.

Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. Omni-math: A universal olympiad level mathematic benchmark for large language models, 2024. URL https://arxiv.org/abs/2410.07985.

Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. Did aristotle use a laptop? a question answering benchmark with implicit reasoning strategies. *Transactions of the Association for Computational Linguistics*, 9:346–361, 2021.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021. URL https://arxiv.org/abs/2103.03874.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL https://arxiv.org/abs/2403.07974.

Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. What disease does this patient have? a large-scale open domain question answering dataset from medical exams. *Applied Sciences*, 11(14):6421, 2021.

Yu Kang, Xianghui Sun, Liangyu Chen, and Wei Zou. C3ot: Generating shorter chain-of-thought without compromising effectiveness, 2024. URL https://arxiv.org/abs/2412.11664.

Xin Lai, Zhuotao Tian, Yukang Chen, Senqiao Yang, Xiangru Peng, and Jiaya Jia. Step-dpo: Step-wise preference optimization for long-chain reasoning of llms, 2024.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *The Twelfth International Conference on Learning Representations*, 2024.

Aiwei Liu, Haoping Bai, Zhiyun Lu, Yanchao Sun, Xiang Kong, Simon Wang, Jiulong Shan, Albin Madappally Jose, Xiaojiang Liu, Lijie Wen, Philip S. Yu, and Meng Cao. Tis-dpo: Token-level importance sampling for direct preference optimization with estimated weights, 2024.

Chenwei Lou, Zewei Sun, Xinnian Liang, Meng Qu, Wei Shen, Wenqi Wang, Yuntao Li, Qingping Yang, and Shuangzhi Wu. Adacot: Pareto-optimal adaptive chain-of-thought triggering via reinforcement learning, 2025. URL https://arxiv.org/abs/2505.11896.

Zimu Lu, Aojun Zhou, Ke Wang, Houxing Ren, Weikang Shi, Junting Pan, Mingjie Zhan, and Hongsheng Li. Step-controlled dpo: Leveraging stepwise error for enhanced mathematical reasoning, 2024.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *arXiv:2308.09583*, 2023.

Wenjie Ma, Jingxuan He, Charlie Snell, Tyler Griggs, Sewon Min, and Matei Zaharia. Reasoning models can be effective without thinking, 2025. URL https://arxiv.org/abs/2504.09858.

MAA Committees. AIME problems and solutions. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, 2024. Accessed: 2025-09-23.

Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward, 2024. URL https://arxiv.org/abs/2405.14734.

Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling, 2025. URL https://arxiv.org/abs/2501.19393.

Sania Nayab, Giulio Rossolini, Marco Simoni, Andrea Saracino, Giorgio Buttazzo, Nicolamaria Manes, and Fabrizio Giacomelli. Concise thoughts: Impact of output length on llm reasoning and cost, 2025. URL https://arxiv.org/abs/2407.19825.

OpenAI. Learning to reason with llms, September 2024. URL https://openai.com/index/learning-to-reason-with-llms/.

Jiahao Qiu, Yifu Lu, Yifan Zeng, Jiacheng Guo, Jiayi Geng, Huazheng Wang, Kaixuan Huang, Yue Wu, and Mengdi Wang. Treebon: Enhancing inference-time alignment with speculative tree-search and best-of-n sampling, 2024. URL https://arxiv.org/abs/2410.16033.

Xiaoye Qu, Yafu Li, Zhaochen Su, Weigao Sun, Jianhao Yan, Dongrui Liu, Ganqu Cui, Daizong Liu, Shuxian Liang, Junxian He, Peng Li, Wei Wei, Jing Shao, Chaochao Lu, Yue Zhang, Xian-Sheng Hua, Bowen Zhou, and Yu Cheng. A survey of efficient reasoning for large reasoning models: Language, multimodality, and beyond, 2025. URL https://arxiv.org/abs/2503.21614.

Qwen Team. Qwen3: A series of large language models. https://qwenlm.github.io/blog/qwen3/, 2024. Accessed: 2025-09-23.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2024.

Zayne Sprague, Xi Ye, Kaj Bostrom, Swarat Chaudhuri, and Greg Durrett. Musr: Testing the limits of chain-of-thought with multistep soft reasoning. *arXiv preprint arXiv:2310.16049*, 2023.

Yang Sui, Yu-Neng Chuang, Guanchu Wang, Jiamu Zhang, Tianyi Zhang, Jiayi Yuan, Hongyi Liu, Andrew Wen, Shaochen Zhong, Hanjie Chen, and Xia Hu. Stop overthinking: A survey on efficient reasoning for large language models, 2025. URL https://arxiv.org/abs/2503.16419.

Chenlong Wang, Yuanning Feng, Dongping Chen, Zhaoyang Chu, Ranjay Krishna, and Tianyi Zhou. Wait, we don't need to "wait"! removing thinking tokens improves reasoning efficiency, 2025a. URL https://arxiv.org/abs/2506.08343.

Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning, 2025b. URL https://arxiv.org/abs/2506.01939.

Yue Wang, Qiuzhi Liu, Jiahao Xu, Tian Liang, Xingyu Chen, Zhiwei He, Linfeng Song, Dian Yu, Juntao Li, Zhuosheng Zhang, Rui Wang, Zhaopeng Tu, Haitao Mi, and Dong Yu. Thoughts are all over the place: On the underthinking of o1-like llms, 2025c. URL https://arxiv.org/abs/2501.18585.

Silei Xu, Wenhao Xie, Lingxiao Zhao, and Pengcheng He. Chain of draft: Thinking faster by writing less, 2025. URL https://arxiv.org/abs/2502.18600.

Shu Yang, Junchao Wu, Xin Chen, Yunze Xiao, Xinyi Yang, Derek F. Wong, and Di Wang. Understanding aha moments: from external observations to internal mechanisms, 2025. URL https://arxiv.org/abs/2504.02956.

Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild, 2025. URL https://arxiv.org/abs/2503.18892.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. Adaptthink: Reasoning models can learn when to think, 2025a. URL https://arxiv.org/abs/2505.13417.

Jintian Zhang, Yuqi Zhu, Mengshu Sun, Yujie Luo, Shuofei Qiao, Lun Du, Da Zheng, Huajun Chen, and Ningyu Zhang. Lightthinker: Thinking step-by-step compression, 2025b. URL https://arxiv.org/abs/2502.15589.

Xiaotian Zhang, Chunyang Li, Yi Zong, Zhengyu Ying, Liang He, and Xipeng Qiu. Evaluating the performance of large language models on gaokao benchmark. *arXiv preprint arXiv:2305.12474*, 2023.

A Related Work

A.1 Over-thinking and Under-thinking

Recent studies have shown that while Large Language Models possess strong reasoning capabilities, they often consume excessive computational resources—a phenomenon known as over-thinking [Qu et al., 2025, Sui et al., 2025]. This primarily manifests in two forms: first, the model's responses contain redundant information; second, it struggles to allocate appropriate computational budgets to questions of varying difficulty, with the latter being particularly evident in O1-like models. To address the first issue, existing solutions include both training-free and training methods. Training-free methods involve carefully designed prompts to limit output length [Xu et al., 2025, Nayab et al., 2025] or adjusting the model's output during decoding to enhance reasoning conciseness [Qiu et al., 2024, Zhang et al., 2025b]. Training methods improve the model's ability to provide concise answers by fine-tuning on concise chain-of-thought (CoT) data or incorporating additional rewards/penalties for output length during reinforcement learning (RL) training [Kang et al., 2024, Aggarwal and Welleck, 2025]. For the second issue, most approaches employ fine-tuning or RL to enable models to dynamically switch between fast-thinking and slow-thinking modes during reasoning based on question difficulty, thereby improving the balance between reasoning accuracy and efficiency [Zhang et al., 2025a, Lou et al., 2025]. Under-thinking represents another form of resource wastage, primarily caused by LRMs frequently switching between reasoning directions before fully developing promising thoughts [Wang et al., 2025c]. Existing methods focus on directly suppressing the model's thought switching behavior at either the token or representation level to mitigate this phenomenon [Ding et al., 2025, Chen et al., 2025a, Wang et al., 2025a]. In contrast, our approach considers that some switching is necessary, and instead alleviates under-thinking by enhancing the model's ability to persist in promising reasoning thoughts.

A.2 Preference Optimization

Preference optimization aims to align model behavior with human or task-specific preferences. For example, some Proximal Policy Optimization (PPO)-based methods typically train a reward model from human preference data and then optimize the policy against this reward while constraining the magnitude of policy updates [Lightman et al., 2024, Luo et al., 2023]. However, such pipelines are typically complex, requiring multiple training stages and additional models. Direct Preference Optimization (DPO) [Rafailov et al., 2024] method simplifies this process by directly optimizing the policy from preference pairs without explicitly training a reward model. Subsequent studies have extended DPO to finer levels of supervision, including step-level and token-level optimization [Lu et al., 2024, Lai et al., 2024, Liu et al., 2024]. Nevertheless, DPO can be sensitive to substantial length differences between preferred and rejected responses because its preference objective is not explicitly normalized by response length. In contrast, Simple Preference Optimization (SimPO) [Meng et al., 2024] addresses this issue by using average log-probability as a length-normalized implicit reward, thereby reducing the influence of response length on preference learning. Inspired by SimPO, we further introduce a finer-grained preference optimization framework, enabling the model to learn thought-level responses to mitigate the under-thinking issue.

B Implementation Details

B.1 Training and Inference Details

For all models, we use a batch size of 256. For the 1.5B model, we use a learning rate of 5×10^{-7}, $\gamma = 1.0$, and $\beta = 2.0$. The 8B, 14B, and 32B models are trained using Low-Rank Adaptation (LoRA). Specifically, the 8B and 32B models use a learning rate of 3×10^{-5} and $\beta = 2.0$, while the 14B model uses a learning rate of 13×10^{-5} and $\beta = 3.0$. The entropy thresholds τ for the 1.5B, 8B, 14B, and 32B models are set to 2.2, 1.3, 1.9, and 1.3, respectively.

Across all experiments, models were optimized using Adam with a context window of 6144. Our training framework incorporated DeepSpeed ZeRO-3 with CPU offload for memory optimization, and we implemented a warm-up ratio of 0.1 with a cosine learning rate scheduler. For all inference tasks, we maintained uniform generation parameters: $max_tokens = 14336$, $temperature = 0.6$, and $top_p = 0.95$.

B.2 Explicit Linguistic Markers

Table 6 lists the predefined linguistic markers used to identify explicit thought switching signals. These phrases typically indicate that the model is reconsidering its current reasoning direction, introducing an alternative approach, or transitioning to a different strategy. During switching-point detection, the linguistic-marker condition is satisfied when the subsequent step begins with one of these predefined phrases.

Table 6: Explicit linguistic markers

alternatively	another way	another thought
another approach	another method	another solution
another technique	think differently	another strategy

B.3 Token Set for Forced Continuation

During the trajectory synthesis stage of ST, we encourage the model to continue along the current reasoning direction by applying a large negative logit bias to tokens that commonly signal a transition to a new thought. The predefined token set used for forced continuation is listed in Table 7.

Table 7: Token set used for forced continuation during trajectory synthesis.

(any	-any	.Any	.any	:Any	:any	<Any	<any	Any
[Any	_any	any	ĠAny	(other	-other	.Other	.other	Other
_other	other	ĠOther	Ġother	Hmm	ĠHmm	Ġhmm	Alternative	alternative
ĠAlternative	Ġalternative	Another	another	ĠAnother	Ġanother	(check	-check	.Check
.check	/check	Check	_Check	_check	check	ĉCheck	ĉcheck	ĠCheck
Ġcheck	Alternatively	ĠAlternatively	Ġalternatively	"But	,but	-but	.But	.but
But	_but	but	âĢ¦but	âĢĶbut	âĢĲBut	ĠBut	Ġbut	However
however	ĠHowever	Ġhowever	.Verify	.verify	Verify	_verify	verify	ĉverify
ĠVerify	Ġverify	Again	_again	again	ĠAgain	Ġagain	(wait	.Wait
.wait	/wait	Wait	_wait	wait	ĉwait	ĠWait	Ġwait	"Now
(now	-now	.Now	.now	Now	[now	_now	now	ĉnow
ĠNow	Ġnow	Ah	ah	aha	ĠAh	Ġah	"Oh	Oh
oh	âĢĲOh	ĠOh	Ġoh	.Maybe	Maybe	maybe	ĠMaybe	Ġmaybe

C Experiments on Different Model Family

To further examine whether the effectiveness of Steady Thought generalizes beyond the Qwen model family, we evaluate ST on DeepSeek-R1-Distill-Llama-8B across multiple reasoning benchmarks. As shown in Table 8, ST consistently reduces token consumption across all evaluated benchmarks. It also improves accuracy on most datasets. These results suggest that the effectiveness of Steady Thought is not limited to Qwen-based reasoning models and can transfer to a different model family.

D Further Analysis of Switching Ability

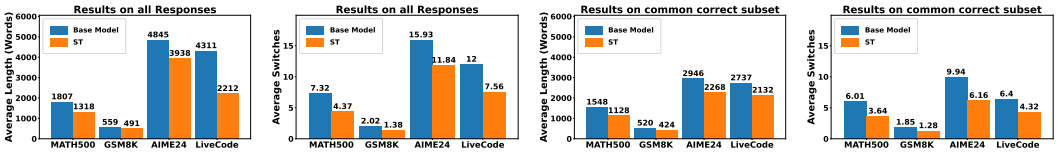
We analyze the variations in thought switching capabilities for the 14B and 32B models before and after the application of ST. As shown in Figure 3, consistent with the results observed in small-scale models, these larger-parameter models demonstrate significantly reduced response length for problems that were already correctly solved by the base models following ST training.

E Ablation on Switching-Point Signals

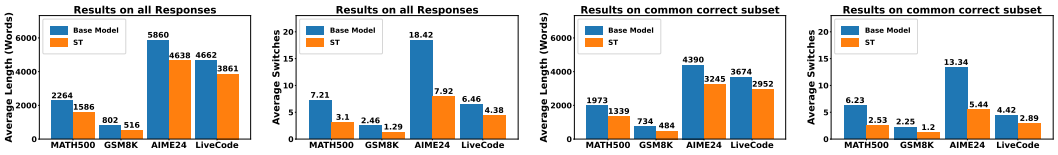
To isolate the contributions of the two signals used for identifying thought switching points, we conduct an ablation study on explicit linguistic markers and boundary entropy spikes. The results are reported in Table 9. The full ST method achieves the best overall trade-off between accuracy and token efficiency, suggesting that the two signals are complementary and that combining them is more effective than relying on either signal alone.

Table 8: Results on DeepSeek-R1-Distill-Llama-8B across different benchmarks.

Method	MATH-500		AIME 2024		GSM8K		LiveCode	
	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓
Vanilla	82.8	3369	41.6	9638	**88.1**	1112	56.4	8398
Steady Thought	85.8	2653	44.2	9100	87.4	874	56.8	7447



(a) DeepSeek-R1-Distill-Qwen-14B



(b) Qwen3-32B

Figure 3: Comparison of switching behavior across four metrics: average response length and number of switches on all responses (first two columns) and on the common correct subset (last two columns). Response length is measured in words. The first row shows DeepSeek-R1-Distill-Qwen-14B, and the second row shows Qwen3-32B. Blue bars represent the base model, and orange bars represent ST.

F Analysis of Different Entropy Thresholds

For threshold calibration, we sample 60 responses per model from the training pool and evaluate candidate thresholds in ascending order based solely on segmentation quality. Specifically, we assess whether the detected boundaries correspond to genuine switches in reasoning direction, using human judgment for clear cases and an LLM judge only for ambiguous ones. We select the lowest candidate threshold for which at least 80% of the detected boundaries are judged to be genuine thought switches. Since increasing τ progressively reduces the number of detected boundaries, this criterion preserves as much boundary coverage as possible while maintaining reliable segmentation.

We investigate the sensitivity of Steady Thought to the entropy threshold τ used for identifying thought switching points. As shown in Table 10, varying τ changes the granularity of thought segmentation and results in different trade-offs between reasoning accuracy and token efficiency. Nevertheless, Steady Thought remains consistently effective across all tested thresholds. For example, on Qwen3-8B, varying τ from 1.0 to 1.6 yields MATH-500 accuracies between 92.6% and 93.4%, all higher than the 91.4% baseline, while substantially reducing the number of output tokens. Similar trends are observed with DeepSeek-R1-Distill-Qwen-1.5B and across the other benchmarks. These results suggest that Steady Thought is not highly sensitive to the specific choice of τ within a reasonable range.

G Details of Data Construction

G.1 Computational Overhead of Data Construction

We provide a detailed discussion on the computational overhead of Steady Thought, focusing on token consumption required for generating the preference data pairs during the trajectory synthesis for various model sizes as shown in Table 11.

It should be noted that, for the 1.5B, 8B, 14B, and 32B models, trajectory synthesis was conducted using only a single 80GB GPU for inference acceleration. We consider this overhead to be acceptable and to yield a high return, particularly given its one-time nature and high efficiency.

Table 9: Ablation study on switching-point signals.

Method	MATH-500		AIME 2024	
	Acc(%)↑	**Tokens↓**	**Acc(%)↑**	**Tokens↓**
DeepSeek-R1-Distill-Qwen-1.5B	82.0	4385	27.5	11273
+ Steady Thought	**83.6**	2691	**30.8**	**7577**
w/o explicit linguistic markers	82.2	**2609**	29.2	8316
w/o boundary entropy spikes	83.4	3554	28.7	8199
Qwen3-8B	91.4	4724	62.1	10895
+ Steady Thought	**93.4**	**2139**	**64.6**	**7640**
w/o explicit linguistic markers	**93.4**	3528	63.3	9712
w/o boundary entropy spikes	93.0	3642	62.9	9685

Table 10: Results under varying thresholds. NT: number of segmented thoughts.

Training model	Threshold	NT	MATH-500		AIME 2024		LiveCode	
			Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓	Acc(%)↑	Tokens↓
DeepSeek-R1-Distill-Qwen-1.5B	-	-	82.0	4385	27.5	11273	30.3	9623
+Steady Thought	1.9	22654	82.4	2987	27.5	7656	32.2	7843
	2.2	15771	83.6	2691	30.8	7577	33.2	7335
	2.5	13338	82.6	2744	30.4	8889	32.8	7853
Qwen3-8B	-	-	91.4	4724	62.1	10895	71.8	7112
+Steady Thought	1.0	33608	92.6	3458	62.9	9623	83.0	6336
	1.3	23188	93.4	2139	64.6	7640	83.2	5631
	1.6	11528	92.6	3548	62.9	9754	84.0	6297

G.2 Data Utilization in Trajectory Synthesis

To better characterize data utilization in the trajectory synthesis stage, we report the number of initial samples, segmented thoughts, and final converted preference pairs in Table 12. For each target model, we start from the same set of initial samples, segment the generated trajectories into thoughts, and then apply forced continuation to identify thoughts that can be successfully completed into correct answers.

We use successful forced continuation as a conservative criterion for determining whether the current thought should be preserved. Specifically, a thought is retained only when the model can continue from it and reach the correct final answer under the forced-continuation setting. Although this criterion may introduce false negatives by discarding some potentially viable thoughts that fail under constrained decoding, it helps ensure the cleanliness and reliability of the resulting preference data. To roughly estimate the false-negative rate, we randomly sample discarded thoughts and ask DeepSeek-V4-Pro to assess whether they remain on a potentially viable reasoning path given the question, reference answer, and reference solution. Approximately 8% of the sampled discarded thoughts are judged to be potentially viable. Since these thoughts are discarded rather than included with incorrect labels, such false negatives primarily reduce data production efficiency rather than introduce noisy supervision into the retained preference data.

After applying this criterion and filtering out overly long instances, the conversion rate from segmented thoughts to final preference pairs is approximately 11.6%–14.2%. Moreover, since a single trajectory may contain multiple correct thoughts, one initial sample can yield multiple preference pairs. As a result, the number of converted pairs corresponds to approximately 167%–195% of the number of initial samples. These statistics show that the synthesis stage provides a substantial amount of usable supervision data in practice, and that potential false negatives do not substantially limit the availability of high-quality thought-level preference pairs.

H Example of Trajectory

We demonstrate the stabilization effect brought by the ST by showing the Qwen3-8B model's response trajectory on the same problem before and after its application as presented in Table 13. We also

Table 11: Token consumption overview for trajectory synthesis stage

Model	Total Tokens Consumed	Average Tokens Consumed
DeepSeek-R1-Distill-Qwen-1.5B	49165922	3117
Qwen3-8B	104958704	6912
DeepSeek-R1-Distill-Qwen-14B	45182779	3374
Qwen3-32B	84345884	5981

Table 12: Data utilization statistics in the Trajectory Synthesis stage.

Model	**Initial Samples**	**Number of Thoughts**	**Converted Pairs**
DeepSeek-R1-Distill-Qwen-1.5B	1051	16177	2054
Qwen3-8B	1051	15185	1757
DeepSeek-R1-Distill-Qwen-14B	1051	13389	1907
Qwen3-32B	1051	15500	1877

illustrate the segmentation results based on entropy. Observing the trajectory reveals that, prior to applying ST, the model frequently switched its thought direction on this problem, ultimately failing to complete the final answer within the pre-set maximum token limit. However, after applying ST, the number of thoughts was reduced from 11 to 4, the token count decreased from 14,338 to 7,413, and the answer was correct.

I Limitations

Our evaluation covers multiple models and datasets, but it is still limited to a finite set of reasoning tasks. Future work could further examine the effectiveness of Steady Thought on broader model families and more diverse application scenarios. In addition, although our switching-point identification strategy is effective in practice, more adaptive detection mechanisms may further improve the efficiency of preference data construction.

J Broader Impacts

This work improves the reasoning efficiency of large reasoning models by reducing redundant token generation while maintaining or improving accuracy. This may help lower inference cost, reduce energy consumption, and make advanced reasoning models more accessible in resource-constrained scenarios. However, since ST is built upon large language models, it may inherit their general risks.

Table 13: An example trajectory before and after ST.

Problem:
The polynomial $x^3 - 3x^2 + 4x - 1$ is a factor of $x^9 + px^6 + qx^3 + r$. Enter the ordered triple (p, q, r).
Reference Answer: (6,31,-1)

BEFORE ST (14336 OUTPUT TOKENS):
Thought 1: Okay, so I need to find the ordered triple (p, q, r) such that the polynomial $x^3 - 3x^2 + 4x - 1$ is a factor of $x^9 + px^6 + qx^3 + r$. ... Let me check if this makes sense.

Thought 2: Wait, let me verify this result by another method to be sure. Maybe by polynomial division or by considering that $x^9 \ldots$ So perhaps my earlier approach is correct.

Thought 3: But let me check with the values I found. If $p = 6, \ldots$ Let me check if $x^3 - 3x^2 + 4x - 1$ divides this.

Thought 4: Alternatively, let me try to perform the division ... So there is a discrepancy in the coefficient of p.

Thought 5: Let me check the first approach again. ... Wait, but this seems conflicting with the previous result.

Thought 6: Wait, actually, no. The original polynomial is $x^9 \ldots$ Let me check my calculations again.

Thought 7: First, let me verify the first approach where I substituted α into the polynomial. ... So the equations are correct.

Thought 8: But then why does the second approach give a different equation for the x^2 coefficient? Wait, maybe the second approach is not correct. ... This seems correct.

Thought 9: The second approach, however, might have an error in the expansion or reduction. Let me recheck ... there is a mistake in one of the approaches.

Thought 10: Wait, let me compute $\alpha^9 + p\alpha^6 + q\alpha^3 + r$ with $p = 6$, $q = 31$, $r = -1$ ... Hence, this approach might not be straightforward.

Thought 11: Alternatively, if I substitute y = x³, then P(x) = y³ + p y² + q y + r. But since x³ = 3x² -4x +1, then y = 3x² -4x +1. Therefore, P(x) = $(3x^2 - 4x + 1)^3 + p(3x - $" ...

AFTER ST (7413 OUTPUT TOKENS):
Thought 1: Okay, so I need to find the ordered triple (p, q, r) such that the polynomial $x^3 - 3x^2 + 4x - 1$ is a factor of $x^9 + px^6 + qx^3 + r$. ... This seems complicated, but maybe there's a smarter way.

Thought 2: Alternatively, since the given polynomial is a factor, then all roots ... Let me check if indeed this is divisible by the cubic.

Thought 3: Alternatively, perhaps there is a better way. ... Let me try this approach.

Thought 4: Let me assume that $Q(x) = x^6 + ax^3 + b. \ldots$ **Final Answer** The ordered triple is $\boxed{(6, 31, -1)}$.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the main contributions of ST, including thought-level preference optimization, token reduction, accuracy improvement, and generalization evaluation, which are supported by the experiments and analyses in the paper.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper includes a limitations section discussing the finite scope of evaluated models and tasks and the efficiency of preference data construction (Appendix I).

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not present formal theoretical theorems or proofs; the mathematical formulations are used to define the problem and optimization objective.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper reports the training data source, evaluation datasets, baselines, hyperparameters, inference settings, and data construction procedure needed to reproduce the main experimental results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We use the open-source datasets and benchmarks, and the code used in this paper will be made publicly available upon acceptance.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specifies the training datasets, evaluation benchmarks, decoding parameters, optimization settings, entropy thresholds, and model-specific training hyperparameters in the main text and appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: We report averaged results over multiple evaluation runs for benchmarks with higher variability, including eight runs for AIME 2024 and two runs for LiveCode. For MATH500 and GSM8K, we use a single run because these datasets contain larger test sets and exhibit relatively stable model outputs, as specified in the experimental protocol.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The paper reports the main training and inference settings and provides additional computational overhead statistics for trajectory synthesis and preference data construction in the appendix.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work uses publicly available models and benchmarks for reasoning research and does not involve private data, human-subject experiments, or procedures that deviate from the NeurIPS Code of Ethics.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses that improving reasoning efficiency can reduce computational cost, while also acknowledging that more efficient reasoning models may inherit the general risks associated with powerful language models (Appendix J).

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release a new high-risk pretrained model, scraped dataset, or generative system requiring additional safeguards beyond those associated with the original public models and datasets.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites the original sources of the models, datasets, and baselines used in the experiments, and these assets are used for research evaluation consistent with their intended academic use.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [N/A]

    Justification: This paper does not release new assets at submission time; the generated preference data and code will be documented and released upon acceptance.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing experiments or research with human subjects.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human subjects, so IRB approval or equivalent review is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The core method directly studies and trains large reasoning models, and the paper describes how these models are used for trajectory generation, forced continuation, and evaluation.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.